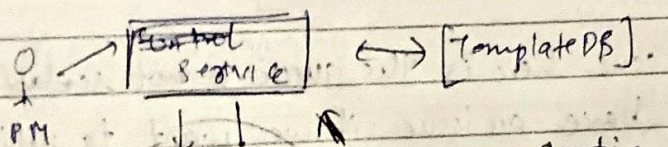


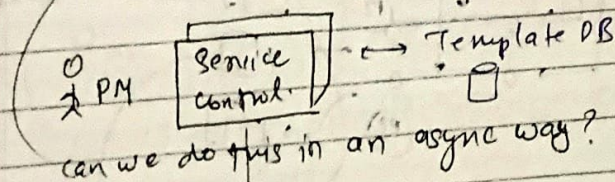
Designing a Notif Service

- sending via all notification channels.
- templates → all templates in a template database
- channels
 - mail, push notifications, message ^{sms}

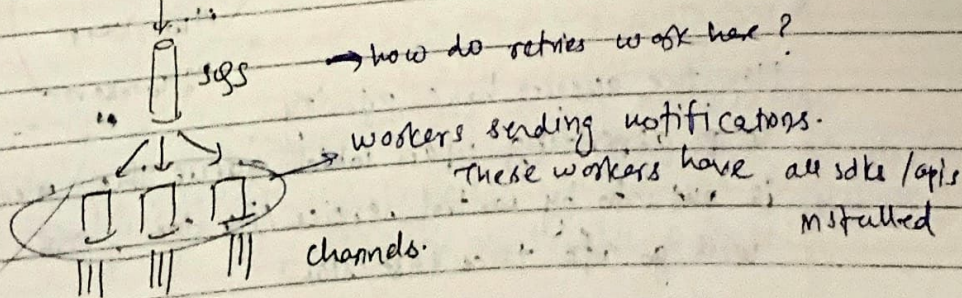
* each thing like mail/msg has a separate api/sdk provider which we can use. Using this → all this msgs can be configured in the service being used to send



Bottlenecks shooting 100K+ directly to consumer through channels & can be really slow. so we need to do this asynchronously.



can we do this in an async way?



→ how do retries work here?

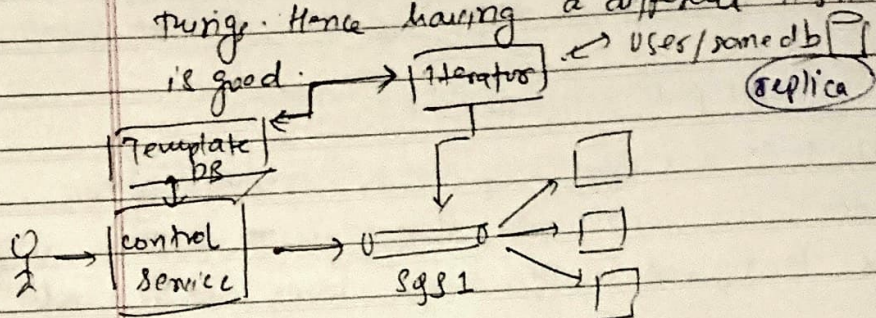
workers sending notifications.

these workers have all sdk/api installed

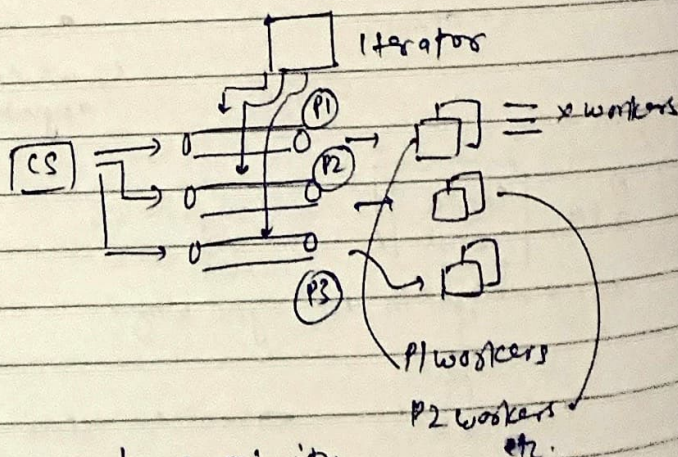
this works for the case, in which we have a list of users on which we want to send data.

→ if we don't have the list for which we wanna iterate upon, and we wish to fanout the notifications to all, or we want to selectively send the notifications to users with a certain criteria → we might find it difficult to do so using CONTROL SERVICE

because it already takes care of analytics & lots of things. Hence having a different Hadoop service is good.

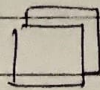


The issue here is the queue is ~~not~~ scalable why? will have an issue if we want to deliver something on priority, here having a single queue - it's ~~too~~ tough to implement priority.

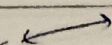


Now the queues have priority, so depending upon which queue the message is put into by control service/iterator, it will go into that fast/slow.

Now, on final caveat, suppose while sending the message to all users, the iterator goes down, we don't want to send the same message again to users who have previously received the message. So what to do?



Iterator



Redis

<user, status>

marketing campaign

For a marketing campaign, we can choose or not choose to send notification to that user, this will have all keys from all users, under certain marketing campaign.

Can we do BETTER?

→ we can have bloom filters which will be space efficient with some percentage of incorrectness.